Pandas & NumPy

Real-World Business Problems, Best Use Cases & Data Analyst Cheat Sheet

Updated Edition — expanded with time-series, window-function, text-processing, outlier-detection, API-ingestion, grain-awareness, and data-quality-measurement techniques, building toward a job-ready Pandas & NumPy analytical foundation.

Reviewer's Note: Is This Enough for a Data Analyst?

Short answer: the original version was a strong foundation but not yet complete. It correctly covered the analyst workflow, core Pandas mechanics (import/export, EDA, cleaning, filtering, grouping, merging), NumPy fundamentals, and validation discipline — it covers a substantial portion of the Python/Pandas workflow used in practical data analysis. (The exact share is impossible to pin to one number: a marketing analyst, a financial analyst, and a BI analyst don't lean on the same subset of tools.)

What was missing (now added in this edition):

- **Time-series intelligence** — rolling averages, period-over-period growth, resampling. Essential for trend and seasonality reporting.
- **Window functions** — cumulative totals and ranking, required for Pareto (80/20) analysis and YTD tracking.
- **Advanced text/CRM cleaning** — regex extraction and pattern matching, since real CRM/transaction data is rarely clean.
- **Outlier detection** — IQR and Z-score methods, so capping (clip) decisions are evidence-based rather than arbitrary.
- **JSON & API ingestion** — modern analysts increasingly pull data directly from APIs, not just flat files.
- **Data grain & metric design** — defining what one row means before aggregating it (Section 18).
- **Data quality KPIs** — measuring a cleaning pass instead of just describing it (Section 19).

These additions close the gap between a "good learning reference" and a "portfolio-ready" cheat sheet. Sections 13-19 below cover each in the same command / business-use format as the rest of the guide.

Where this fits in the bigger picture: Pandas and NumPy are one stage in a longer pipeline, not the whole of business analytics. A typical flow looks like: *business question → SQL (extraction) → Pandas/NumPy (cleaning & analysis) → statistics (validating the insight) → a BI tool like Power BI/Tableau (communicating it) → business decision.* This document strengthens the middle of that chain — it isn't a substitute for the SQL, statistics, or BI layers on either side of it.

1. Real-World Business Problems & Best Use Cases

Customer Lifetime Value (CLV) & Cohort Analysis

Segment customers by first purchase date and calculate Recency, Frequency, and Monetary (RFM) value. Useful for identifying valuable customers, churn patterns, cohort quality, and profitable acquisition spend.

Sales Pipeline & Funnel Optimization

Track deal stages, pipeline revenue, products, regions, managers, and representatives. `pivot_table()` is useful for rapidly summarizing deal status and revenue.

Financial Cash-Flow Modeling

Replace repetitive spreadsheet scenario calculations with vectorized Pandas/NumPy operations where appropriate. Benchmark before claiming performance improvements.

Automating Repetitive Business Reports

Automate extraction, cleaning, matching, metric calculation, and delivery to BI/reporting systems. This is where Python becomes a workflow tool rather than merely a notebook language.

Missing Data at Scale

Audit missingness before changing data. Drop invalid records when appropriate, fill gaps when justified, and use segment-aware or statistical imputation when the business context supports it.

2. Pandas Cheat Sheet: Essential Commands

Data Import & Export

Command	Business use
`pd.read_csv('file.csv')`	Load CSV data.
`pd.read_excel('file.xlsx', sheet_name='Sheet1')`	Read a specific Excel sheet.
`pd.read_sql("SELECT ...", con=engine)`	Load SQL query results into a DataFrame.
`df.to_csv('clean_data.csv', index=False)`	Export a clean tabular dataset.
`df.to_parquet('clean_data.parquet', index=False)`	Efficient columnar storage for analytical pipelines.
`df.to_feather('data.feather')`	Fast local binary storage for prototyping.

Inspection & EDA

Command	Business use
`df.head(n) / df.tail(n)`	Inspect beginning/end of data.
`df.info()`	Check row counts, nulls, dtypes, and memory usage.

`df.describe()`	Generate numerical descriptive statistics.
`df.shape`	Get row/column dimensions.
`df.dtypes`	Inspect column data types.
`df.nunique()`	Check cardinality of columns.
`df['column'].value_counts()`	Count categorical values.

Cleaning & Missing Values

Command	Business use
`df.isnull().sum()`	Audit missing values by column.
`df.dropna(subset=['col'])`	Drop records missing a critical field.
`df['col'] = df['col'].fillna(value)`	Fill missing values.
`df.drop_duplicates()`	Remove exact duplicate records.
`pd.to_numeric(df['col'], errors='coerce')`	Convert messy numeric text safely.
`pd.to_datetime(df['date'], errors='coerce')`	Convert dates for time analysis.

Filtering & Subsetting

Command	Business use
`df[['col1', 'col2']]`	Select multiple columns.
`df.iloc[0:5, 0:3]`	Position-based selection.
`df.loc[df['age'] > 30, ['name','age']]`	Label-based selection plus filtering.
`df.query("age > 30 and sales >= 12000")`	Readable conditional filtering.

Wrangling, Grouping & Reshaping

Command	Business use
`df.groupby('category_col').agg({'numeric_col':'mean'})`	Split-apply-combine aggregation.
`pd.pivot_table(df, index='Region', columns='Product', values='Sales', aggfunc='sum')`	Multi-dimensional business summary.
`pd.melt(df)`	Convert wide data to long/tidy form.
`pd.merge(df1, df2, on='customer_id', how='inner')`	Database-style join.

`pd.concat([df1, df2], axis=0)`	Append compatible datasets by rows.

3. Data Cleaning & Production-Grade Wrangling

Tidy data principle: each variable should normally be a column and each observation a row. This makes vectorized operations, grouping, visualization, and downstream modeling easier.

Standardizing Column Headers

Clean whitespace, casing, and separators before downstream transformations.

```
df.rename(columns=lambda x: x.strip().lower().replace(' ', '_'), inplace=True)
```

Method Chaining

Use chaining when it makes a transformation pipeline readable and auditable. Do not chain merely to look clever.

```
cleaned_report = (
    sales
    .rename(columns=lambda x: x.strip().lower().replace(' ', '_'))
    .assign(profit_margin=lambda x: (x.total - (x.units * x.unit_cost)) / x.total)
    .query('units > 10')
    .groupby('region')['total'].sum()
)
```

Important Validation Checks

- Check row counts before and after major transformations.
- Check uniqueness of business keys before merging tables.
- Check duplicate rows after merges because many-to-many joins can multiply records.
- Check whether numeric columns contain strings or invalid values.
- Check date ranges and chronological order.
- Compare important totals before and after transformation to detect accidental inflation or loss.
- Document assumptions, especially around missing values, returns, cancellations, and duplicates.

4. Business Use Case: Sales Performance

Regional Revenue Share

Use grouped revenue divided by total revenue to calculate regional contribution. This supports questions such as which region dominates and whether a region is underperforming.

```
df.groupby('region')['total'].sum() / df['total'].sum() * 100
```

Top Sales Representatives

```
df.groupby('rep')['total'].sum().sort_values(ascending=False)
```

Inventory / Product Popularity

```
df['item'].value_counts()
```

Temporal Trends

```
df['month'] = df['orderdate'].dt.month
```

5. Customer Lifetime Value (CLV) & RFM

Strategic questions: Which customers are most valuable? Which acquisition cohorts perform best? Which customers are becoming inactive? How much can the business afford to spend to acquire a customer?

- **Descriptive CLV:** establish average and distribution of customer value.
- **Cohort CLV:** compare customers by acquisition/first-purchase month.
- **RFM:** Recency, Frequency, Monetary segmentation.
- **Predictive CLV:** use probabilistic or machine-learning models only after the descriptive data foundation is reliable.

6. Advanced Aggregation & Dataset Combination

Command	Business use
`Left merge`	Attach customer/product metadata to a transaction table while preserving transactions.
`Inner merge`	Keep records with matching keys in both datasets.
`Multiple-key merge`	Match records using a compound business key such as customer_id + order_date.
`Filtering with isin()`	Keep records whose IDs exist in another reference/list.
`concat(axis=0)`	Append compatible monthly or regional files.

Merge Validation Checklist

- Define the expected relationship: one-to-one, one-to-many, or many-to-one.
- Verify the join key's uniqueness where uniqueness is expected.
- Inspect row counts before and after merging.
- Check duplicated business records after the merge.
- Compare totals before and after the merge.
- Use Pandas merge validation options when appropriate, such as validate='one_to_many'.

7. NumPy Cheat Sheet

Array Creation & Inspection

Command	Business use
`np.array()`	Create a NumPy array.
`np.zeros((3,3)) / np.ones((2,2))`	Create initialized arrays.
`np.linspace(0, 1, 5)`	Generate evenly spaced numerical values.
`arr.shape / arr.ndim / arr.dtype`	Inspect structure and data type.

Vectorized Mathematics

Command	Business use
`arr * 2 / arr + 5`	Element-wise arithmetic without explicit loops.
`np.dot(A, B) / A @ B`	Dot product / matrix multiplication.
`np.sin(arr) / np.exp(arr) / np.log(arr)`	Apply mathematical functions element-wise.

Statistics

Command	Business use
`np.mean(arr) / np.median(arr)`	Central tendency.

`np.std(arr) / np.var(arr)`	Dispersion.
`np.sum(arr, axis=0)`	Aggregate down columns in a 2D array.
`np.sum(arr, axis=1)`	Aggregate across rows in a 2D array.
`np.max(arr) / np.min(arr)`	Extremes.

Reshaping

Command	Business use
`arr.reshape(rows, cols)`	Change dimensional representation without changing elements.
`arr.T`	Transpose rows and columns.
`arr.flatten() / arr.ravel()`	Convert multi-dimensional data to 1D.

8. Data Visualization Support

- `df.plot.scatter(x='units', y='total')` — Explore relationships such as units sold vs. revenue.
- `df.plot.barh()` — Compare sales representatives, regions, or categories.
- `df.plot.hist()` — Inspect the distribution of transaction values.

Important: visualization should answer a question. Do not create charts merely because the plotting API exists. For polished portfolio work, use dedicated BI tools when interactivity and stakeholder reporting are required.

9. Financial / Numerical Controls

- `df.max(axis=1)` — Find the maximum value across columns for each row.
- `df.clip(lower, upper)` — Cap extreme values when the business rule genuinely requires it.
- `df.abs()` — Calculate absolute errors/deviations.
- `np.mean(), np.median(), np.std()` — Establish numerical baselines for distributions and variation.

10. Final Best Practices

- **Prototype locally, then scale:** Pandas/NumPy are excellent for local analytical workflows; larger data may require SQL, Spark, warehouses, or distributed tools.
- **Validate before trusting:** successful execution does not mean correct analysis.
- **Impute with context:** median, mean, forward-fill, or deletion should be justified by the data-generating process and business meaning.
- **Prefer vectorization:** use built-in/vectorized operations where practical instead of unnecessary Python loops.
- **Use method chaining carefully:** prioritize readability and auditability.
- **Keep transformations reproducible:** avoid manual spreadsheet edits that cannot be rerun.
- **Separate exploration from production:** notebooks are excellent for discovery; reusable logic should eventually move into scripts/modules.
- **Machine-learning readiness:** prepare clean numerical and categorical features only after understanding the business semantics.

11. Analyst Mental Model

Never start with the Pandas function. Start with the business question and the level of the entity being analyzed.

Business question → define metric → identify grain → inspect data → clean → combine → aggregate → validate → analyze → explain → recommend

Example: “Which product performs best?” is incomplete. Best by what: revenue, units, margin, repeat purchases, return-adjusted revenue, or average order value? The definition determines the aggregation level and therefore the correct Pandas workflow.

12. Portfolio Application

These skills are intended to feed directly into a real business case such as an **E-commerce Revenue Leakage Audit**: multiple tables → data-quality audit → Pandas cleaning → merge validation → revenue/discount/return/payment analysis → business recommendations.

Rule for portfolio projects: every major metric should have a clear definition, reproducible calculation, validation check, and business interpretation.

NEW IN THIS EDITION

13. Time-Series Intelligence

Business questions this unlocks: Is the trend accelerating or decelerating? How does this month compare to last month or last year? What does a smoothed (noise-free) view of daily sales look like?

Command	Business use
`df.set_index('date').resample('M').sum()`	Aggregate daily/transaction data into monthly (or weekly/quarterly) totals.
`df['sales'].rolling(window=7).mean()`	7-day rolling average to smooth out daily noise and reveal trend.
`df['sales'].pct_change()`	Period-over-period growth rate (e.g., day-over-day, month-over-month).
`df['sales'].pct_change(periods=12)`	Year-over-year growth when data is monthly.
`df['sales'].shift(1)`	Create a lagged column to compare current vs. prior period side by side.
`df.groupby(df['date'].dt.to_period('M'))['sales'].sum()`	Business-calendar aware monthly grouping using Period objects.

Watch-out: rolling and pct_change require the data to be sorted chronologically and, ideally, on a continuous date index. Reindex to a full date range first if the transaction log has gaps.

14. Window Functions: Cumulative Totals & Ranking

Business questions this unlocks: Which 20% of products drive 80% of revenue (Pareto)? What is year-to-date revenue as of any given day? How do reps rank against each other within their region?

Command	Business use
`df['cumulative_sales'] = df['sales'].cumsum()`	Running total — the basis for YTD tracking and Pareto (80/20) charts.
`df['pct_of_total'] = df['cumulative_sales'] / df['sales'].sum()`	Cumulative share of total, used to find the 80% cutoff point.
`df['rank'] = df['sales'].rank(ascending=False, method='dense')`	Rank products/reps by performance, handling ties explicitly.
`df.groupby('region')['sales'].rank(ascending=False)`	Rank within each group (e.g., rep rank inside their own region).
`df.groupby('region')['sales'].transform('sum')`	Broadcast a group aggregate back onto every row without collapsing the DataFrame.
`df.groupby('customer_id')['order_date'].cummax()`	Track the most-recent order seen so far per customer, row by row.

`transform()` is the most underused method here: it lets you add a "% of region total" or "rank within group" column while keeping the original row-level grain — exactly what's needed for dashboards.

15. Advanced Text Processing for Messy CRM Data

Business questions this unlocks: How do I pull a product code, invoice number, or area code out of a free-text field? How do I flag records that mention a competitor, a complaint, or a specific keyword?

Command	Business use
`df['col'].str.extract(r'(\d{5})')`	Regex extraction — e.g., pull a 5-digit ZIP or ID code out of a free-text field.
`df['col'].str.contains('refund', case=False, na=False)`	Flag rows matching a keyword/pattern; na=False avoids errors on missing text.
`df['col'].str.replace(r'\s+', ' ', regex=True)`	Collapse irregular whitespace/line breaks in imported text.
`df['col'].str.split('-', expand=True)`	Split a compound field (e.g., 'REGION-PRODUCT-ID') into separate columns.
`df['col'].str.strip().str.title()`	Standardize free-text names/labels for consistent grouping.
`df['email_domain'] = df['email'].str.extract(r'@(.+)$')`	Derive a new business field (e.g., company domain) from a text column.

Validation habit: after any `str.extract()`, run `.isnull().sum()` on the result — a spike in NaNs usually means the regex doesn't match a common real-world variant of the pattern.

16. Outlier Detection: IQR & Z-Score

Business questions this unlocks: Which transactions are statistically unusual and worth a manual review? Is a "clip" threshold defensible, or arbitrary? Should this outlier be treated as fraud, a data-entry error, or a genuine large deal?

Interquartile Range (IQR) method — robust to skewed business data:

```
Q1 = df['sales'].quantile(0.25)
Q3 = df['sales'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = df[(df['sales'] < lower) | (df['sales'] > upper)]
```

Z-Score method — better for roughly normal distributions:

```
from scipy import stats
df['z_score'] = stats.zscore(df['sales'])
outliers = df[df['z_score'].abs() > 3]
```

Rule of thumb: use IQR when the metric is skewed (most revenue/transaction data is), and reserve Z-score for metrics that are approximately normally distributed. Either way, flag and investigate before using `df.clip()` — capping should follow evidence, not precede it.

17. JSON & API Data Ingestion

Business questions this unlocks: How do I pull data directly from a marketing, CRM, or product analytics API instead of waiting for a manual export? How do I flatten nested JSON responses into an analyst-friendly table?

Command	Business use
`pd.read_json('data.json')`	Load a JSON file directly into a DataFrame.

`pd.json_normalize(response.json()['results'])`	Flatten a nested JSON API response into a flat, tabular DataFrame.
`requests.get(url, headers=headers).json()`	Pull data directly from a REST API (used with the requests library).
`df.to_json('export.json', orient='records')`	Export a DataFrame back to JSON, e.g., for a downstream API or app.

Production habit: API responses are rarely flat — always inspect one record's structure (`response.json()['results'][0]`) before calling `json_normalize()`, and paginate/rate-limit responsibly against the source API.

18. Data Grain & Metric Design

This is the step that should happen **before** a single line of Pandas code is written, and it's the difference between a correct metric and a confidently wrong one.

Grain: what does one row represent?

Every table has a grain — the real-world entity one row corresponds to. Common grains in business data:

- 1 row = 1 order
- 1 row = 1 order line item
- 1 row = 1 customer
- 1 row = 1 payment
- 1 row = 1 product

The grain determines whether an aggregation is even asking the right question. For example:

Command	Business use
`df['sales'].mean()`	Average transaction value — grain is one row per transaction.
`df.groupby('customer_id')['sales'].sum().mean()`	Average revenue per customer — a completely different metric, despite reusing 'sales' and 'mean'.

Rule of thumb: before writing `.groupby()` or `.mean()`, state out loud what one row means *right now* and what one row will mean *after* the aggregation. If you can't state both, the metric isn't defined yet — and a ranking or rate calculated at the wrong grain will look plausible while being wrong.

19. Data Quality KPIs

"I cleaned the data" is not a finding — it's an unmeasured claim. A portfolio-grade cleaning pass reports what changed and why, the same way a financial report reconciles a balance.

Example cleaning log:

Metric	Count
Rows before cleaning	`50,000`
Rows after cleaning	`49,731`
Duplicate rows removed	`184`
Missing customer IDs	`61`
Invalid dates	`24`
Invalid revenue values	`17`

Command	Business use
`len(df)`	Row count — capture before and after every major cleaning step.
`df.duplicated().sum()`	Count exact duplicate rows before dropping them.
`df['col'].isnull().sum()`	Count missing values per critical field.

`valid_rate = len(df_clean) / len(df_raw)`	Data quality rate: the share of records that survived cleaning intact.

Why this matters: a stated data-quality rate (e.g., "99.5% of records were valid; the remaining 0.5% were missing a customer ID and were excluded, not silently dropped") reads as professional diligence. An unmeasured "I cleaned it" reads as a guess.

20. Updated Skill Coverage Checklist

Use this as a self-assessment before applying to Data Analyst roles. If you can explain the business reason for every row below — not just the syntax — this cheat sheet now covers a job-ready foundation.

- Data import from files, databases, and APIs (Sections 2, 17)
- Structured EDA and data-quality auditing (Sections 2, 3)
- Cleaning: missing values, duplicates, type coercion, text/regex cleanup (Sections 2, 3, 15)
- Filtering, grouping, pivoting, and reshaping (Sections 2, 6)
- Merging/joining with validation discipline (Sections 3, 6)
- Time-series analysis: trends, growth rates, rolling metrics (Section 13)
- Window functions: cumulative totals, ranking, Pareto analysis (Section 14)
- Outlier detection grounded in statistics, not guesswork (Section 16)
- Core NumPy vectorization and array statistics (Section 7)
- Business framing: CLV/RFM, sales performance, revenue-leakage audits (Sections 1, 4, 5, 12)
- An analyst mental model that starts from the business question, not the function (Section 11)
- Grain-aware metric design — knowing what one row means before aggregating it (Section 18)
- Measuring a cleaning pass with data-quality KPIs instead of just describing it (Section 19)

Still worth building separately (outside Pandas/NumPy): SQL for warehouse-scale querying, a BI tool (Power BI / Tableau / Looker) for stakeholder-facing dashboards, and enough statistics to defend a hypothesis test or confidence interval in an interview. Pandas/NumPy sit in the middle of that pipeline — they don't replace either side of it.

A note on scope: this document has reached the point of diminishing returns as a reference — more commands from here add clutter, not capability. The next improvement doesn't come from expanding this PDF further; it comes from applying everything above to a single messy, multi-table, realistic dataset (e.g., an end-to-end revenue-leakage investigation) where cleaning, merging, aggregation, validation, time analysis, and business interpretation all have to work together. The PDF is the reference; the project is the proof.